案例 04 · crm-cleaner：客户主数据清洗与去重

角色：独立完成（建模 → 编码 → 测试 → 文档 → 开源）

时间：2026.10

技术：Python 3.11+（零第三方依赖）· csv / difflib / tomllib / unittest · GitHub Actions

可验证材料：`github.com/Cix1125/crm-cleaner`（公开仓库，CI 通过）

规模：22 个文件 · **1195 行 Python**（核心 855 + 测试 340）· 35 个测试用例

一、问题（这类项目里最先卡住人的地方）

CRM 系统上线、数据迁移的时候，客户数据永远是脏的。我在跟客户打交道的过程中反复看到同一批毛病：

- **同一个客户，几个写法**：`郑州云启科技有限公司` / `郑州云启科技` / `郑州云启科技（郑州）`
- **格式不一致**：邮箱大小写混用（`ZhangWei@YunQi.com`），电话带横杠、空格、括号、国家码
- **不同系统导出的表，列名不一样**：CRM 里叫「公司名称」，Excel 台账里叫「客户名称」，客服系统里叫「客户全称」
- **合并完了说不清**：迁移做完，没人能回答「到底合并了哪些、改了哪个字段」

现在普遍的做法是**拿 Excel 一行行看**。数据量一大就没人敢动了——因为改错了没人兜底。

二、我做了什么

1. 先定"认领字段"这件事，再定判重

这是我做的第一个判断：**不能假设各来源的列名一致。**

规则文件里给每个逻辑字段配一串候选名，工具**按文件分别认领**：

```
[columns]
company = ["公司名称", "客户名称", "公司", "company", "company_name"]
email   = ["邮箱", "电子邮箱", "email", "e-mail"]
phone   = ["电话", "手机", "联系电话", "phone", "tel", "mobile"]
```

> **这个判断后来救了我一次。** 第一版我只按第一个文件的表头认领，结果第二份文件（列名不同）的数据归一化后全是空字符串，**判重结果是 0 组**。这个 bug 恰好证明了这个工具存在的理由。

2. 判重分三层，从"绝对可靠"到"可能相似"

层	规则	为什么
1 · 精确键	归一化后的邮箱或电话完全相同	最可靠，几乎不误判
2 · 规范化	公司名去掉「有限公司 / Ltd. / Inc.」等后缀后相同	抓写法差异
3 · 相似度	`difflib` 相似度 ≥ 0.88	兜底——`洛阳新材` vs `洛阳新材料` = 0.889

单比一层都会出事：只比邮箱会漏掉没填邮箱的；只比公司名抓不到后缀差异；只比相似度误合并率高、还慢。

3. 用并查集处理"重复的传递性"

重复不是一对一的：**A 和 B 邮箱相同、B 和 C 公司名相同 → A、B、C 是同一家。**

用并查集（union-find）合并，并加路径压缩：

```
def find(self, x: int) -> int:
    root = x
    while self._parent[root] != root:
        root = self._parent[root]
    while self._parent[x] != root:          # 路径压缩
        self._parent[x], x = root, self._parent[x]
    return root
```

4. 前两层用分桶，第三层才两两比

前两层把 N 条记录按 key 塞进字典，同桶即重复 —— **O(N)** 而不是 O(N²)。

只有相似度那一层躲不掉两两比较，所以加了**首字预过滤**：公司名首字不同的直接跳过。

5. 变更日志 —— 我认为这个工具最值钱的部分

迁移完必须能回答"到底动了什么"，否则数据团队不敢用。所以每一处改动都记下来：

```
组内记录数,字段,动作,原值,新值,取值来源,主记录来源
2,city,补空,,洛阳,customers_b.csv#4,customers_a.csv#4
```

设计上有一条硬规矩：原始值一个字都不美化。 工具的职责是合并重复，不是改写内容。

6. 配套：35 个测试 + 中英双文档 + CI

- `tests/` 覆盖归一化规则、并查集传递性、三层匹配、CLI 端到端、GBK 编码、跨文件列名不一致
- `README.md` + `README.en.md`
- GitHub Actions 在 3.11 / 3.12 / 3.13 三个版本上跑测试，**当前全绿**

三、结果

指标	值
代码规模	**1195 行 Python**（核心 855 + 测试 340）
测试用例	**35 个，全部通过**
第三方运行时依赖	**0 个**
CI	GitHub Actions，3 个 Python 版本，**通过**
示例效果	10 条记录 → 发现 4 组重复 → 合并成 6 条，且**没有误合并无关公司**
是否有人用	公开仓库，个人项目，**未在生产环境使用**

四、这个案例能证明我什么

- **工程判断力**：知道"单比一个字段会出事"，所以分层；知道"阈值定高了漏、定低了误合并"，所以用测试兜住
- **数据结构基础**：并查集 + 路径压缩，处理传递性关系
- **性能意识**：能 O(N) 的不写 O(N²)，躲不掉的加预过滤
- **对"交付"的理解**：主动做变更日志，因为知道客户真正怕的是"改了说不清"
- **文档能力**：中英双 README + 规则注释，让不写代码的人也能看懂
- **能把工具用起来**：会用 AI 辅助写代码、查文档，也清楚哪些判断必须自己做

五、我知道自己的边界

- **没有模糊匹配以外的智能。**相似度阈值永远需要人工过一遍，工具只是把工作量从"逐条看"降到"看报告"
- **只支持 CSV。**Excel 要先另存为 CSV
- **相似度是字符级的。**`郑煤机` ↔ `郑州煤矿机械集团` 这种简称关系抓不到
- **没有做性能优化。**第三层是 O(N²)，几万条还行，十万级以上要换方案
- **这是个个人项目，没在真实生产环境跑过。**示例数据是虚构的
- 开发过程中用了 AI 辅助（写代码、查文档），但**设计判断、规则取舍、测试用例是我定的**——我知道哪部分该自己拿主意

六、面试官可能追问（提前想好）

1.「你这个跟现成的数据清洗工具有什么区别？」

> 现成工具功能全但重，要部署、要配置、要培训。我做的只解决一件事——**多源客户名单的判重**，而且零依赖，`git clone` 下来就能跑。数据迁移经常在客户内网做，装包要走审批，这是它的价值。
>
> 诚实补一句：如果客户的场景更复杂（要接数据库、要跑调度），我会推荐直接上成熟的数据平台。

2.「相似度阈值为什么是 0.88？」（高频，必须答得出）

> 算出来的，不是拍的。`洛阳新材` 和 `洛阳新材料` 用 difflib 算出来是 **0.889**——阈值定 0.9 就漏掉它；再往低就会把 `河南云启信息技术` 和 `郑州云启科技` 这种只共前缀的无关公司误合并。0.88 夹在中间，而且这两个方向我都写了测试。

3.「为什么用并查集？」

> 因为重复是传递的。A 和 B 邮箱相同、B 和 C 公司名相同，三个就是同一家。朴素的递归合并要处理"已经被合并过"的状态，很乱；并查集三行解决，加路径压缩后近似 O(1)。

4.「你这个项目代码是 AI 写的吗？」（一定会问，别躲）

> 用了 AI 辅助，主要是写样板代码和查文档。但有几件事是我自己定的：**三层匹配怎么分层、阈值定多少、变更日志要记哪些字段、保留原始值不改写。**而且 35 个测试用例是我一条条列的——每一层至少一个正例一个反例。
>
> **我不会硬撑说全是我手写的，也不会把 AI 写的说成自己的想法。**

5.「你这个项目最大的局限是什么？」

> 相似度那层永远需要人工过一遍。工具解决的是"把工作量从逐条看降到看报告"，不是"完全自动判断"。
>
> 还有个更根本的：它只能算字符相似，不懂业务关系。`郑煤机` 和 `郑州煤矿机械集团` 是同一家，但它抓不到——要解决得上别名库。

6.「为什么不选 pandas？」

> pandas 处理这种几千行、字段规整的数据是杀鸡用牛刀，而且它是个重依赖。用标准库做完之后，这个工具在任何装了 Python 的机器上都能跑，包括客户内网。**零依赖是刻意选的，不是不会用 pandas。**

7.「你怎么保证没把不该合并的合并了？」

> 三层里第 3 层是唯一可能误判的，所以我在 HTML 报告里标出了每一组是第几层匹配出来的。第 3 层的组必须人工过一遍。另外主数据里留了 `_来源` 字段，能追到原始文件第几行。

8.「如果要你把它做成生产级，你要加什么？」

> 三件：一是别名库（把简称、集团简称维护进去）；二是写回能力——生成"合并操作清单"让 CRM 直接执行，而不是只给人看报告；三是性能——第三层现在是 $O(N^2)$，几万条以上得换索引方案（比如先按拼音首字母分桶）。